

ALMA MATER STUDIORUM · UNIVERSITÀ DI BOLOGNA

Second Cycle Degree
Artificial Intelligence

Exercises
Introduction to
Complexity and Computability

Student:
Matteo Canghiari

Academic Year 2025/2026

Chapter 1

Conceptual and mathematical preliminaries

...

Chapter 2

Computational model

Undecidability halting problem

Problem.

Prove the undecidability of the **halting problem**, namely the fact that the following function

$$halt(\alpha, x) = \begin{cases} 1 & \text{if } M_\alpha(x) \text{ terminates} \\ 0 & \text{otherwise} \end{cases}$$

is **not computable**.

Note.

For this kind of problem, we have two different ways to solve it:

- **reduction**, reducing the proof to a well-known existing problem already uncomputable;
- **directly**, solving it.

We use a technique that lies between the ones presented, exploiting a well-known problem that is already undecidable (*namely the uc function*).

Observations.

- We want to prove that the computational task *halt* is undecidable.
- The main assumption of the proof is that we know for sure that it's impossible to build a machine that computes *halt*.
- We start saying that exists a Turing Machine able to compute the *halt* function and then we get a contradiction about our original thesis.

Solution.

We show that from a hypothetical machine computing the function $halt$, call it M_{halt} , we can build another machine M_{uc} which computes function uc . Since we know such a machine cannot exist (*by definition the uc function is uncomputable*), we are done.

Described the main idea, we need to use M_{halt} as a subroutine in building M_{uc} , so the $halt$ function is an inner piece of M_{uc} in our construction of the function uc .

Let's construct M_{uc} using M_{halt} as a subroutine.

On input α the machine M_{uc} proceeds as follows:

1. We call M_{halt} on input $(\alpha, \alpha) \in \{0, 1\}^*$, where x is equal to α .
2. We wait until M_{halt} returns the output.
3. We proceed in two different ways than the output returned by M_{halt} :
 - (a) If M_{halt} returns 0 on input (α, α) , meaning that $M_\alpha(\alpha)$ does not terminate (*we are in the second case of uc where M_{uc} returns 1*).
 - (b) If M_{halt} returns 1 on input (α, α) , M_{uc} knows that it can safely call U , the Universal Turing Machine, on input (α, α) , since $M_\alpha(\alpha)$ terminates and thus $U(\alpha, \alpha)$ terminates too. Therefore, let's then call $U(\alpha, \alpha)$ and let b be the answer we receive (*we are using U as M_α , since $U(x, \alpha) = M_\alpha(x)$ by definition*).
4. We return the negation of b , as follows:
 - (a) If $b = 1$, then $M_\alpha(\alpha) = 1$, and $uc(\alpha) = 0$ by definition.
 - (b) If $b = 0$, then $M_\alpha(\alpha) = 0$, and $uc(\alpha) = 1$ by definition.
5. The way we have defined M_{uc} ensures that it computes the function uc , in particular relying on the fact that M_{halt} solves the computational task $halt$ and U is the Universal Turing Machine.

Final mark.

Since such machine M_{uc} does not exist, also M_{halt} does not exist in the first place, therefore we have done. $\square$

Note.

Why do we introduce the Universal Turing Machine U inside the proof?

Giving in input (α, α) to the Turing Machine M_{halt} tells us only if the computation stops or proceeds, it doesn't give us the exact result. Therefore, when the computation finally stops, M_{halt} does not give us any information about the result. So, we need the Universal Turing Machine in order to simulate M_{halt} over α to achieve the final answer, that in this case it could be $y \in \{0, 1\}$.

Haltstar problem

Problem.

Given the following function

$$\text{haltstar}(s) = \begin{cases} 1 & \text{if } s = \lfloor \alpha, x_1, \dots, x_n \rfloor \wedge M_\alpha(x_1) \wedge \dots \wedge M_\alpha(x_n) \text{ terminates} \\ 0 & \text{if } s = \lfloor \alpha, x_1, \dots, x_n \rfloor \wedge \exists i \in \{1, \dots, n\} \text{ s.t.} \\ & M_\alpha(x_i) \text{ does not terminate} \end{cases}$$

is it computable or uncomputable?

Note.

- We do not check the termination condition only on one input, but over many inputs $x_1, \dots, x_n$.
- We name L_{haltstar} the language underlining the *haltstar* function, putting in relation the problem L_{haltstar} with an already known undecidable problem.

Observations.

- The whole thing is just being able to frame the problem in the following way:

$$s \in G \rightarrow \phi(s) \in L$$

Solution.

We prove *haltstar* to be **uncomputable**, showing L_{haltstar} is undecidable (L_{haltstar} is the set of strings on which *haltstar* returns 1). The latter can be proved by reduction from the undecidability of L_{halt} (reducing the proof to a problem that we already know to be uncomputable).

We have to define, in other words, a function as follows

$$\phi : \{0, 1\}^* \rightarrow \{0, 1\}^*$$

such that two constraints are satisfied:

1. ϕ is computable.
2. $s \in L_{\text{halt}} \Leftrightarrow \phi(s) \in L_{\text{haltstar}}$

We define ϕ as follows:

$$\lfloor (\alpha, x) \rfloor \xrightarrow{\phi} \lfloor \alpha, x \rfloor$$

so, ϕ it's just an identity function, a way to encode pairs to tuples. Anyway, the encoding done by this function it's a little bit different than what we have seen so far, in particular:

- $\lfloor (\alpha, x) \rfloor \rightarrow$ encoding of pairs
- $\lfloor \alpha, x \rfloor \rightarrow$ encoding of tuples

We can state:

- ϕ is easily seen to be computable, it does nothing besides playing with encoding of pairs and tuples.
- moreover

$$\begin{aligned}
s \in L_{\text{halt}} &\Leftrightarrow s = \lfloor (\alpha, x) \rfloor M_{\alpha}(x) \text{ terminates} \\
&\Leftrightarrow \phi(s) = \lfloor \alpha, x \rfloor M_{\alpha}(x) \text{ terminates} \\
&\Leftrightarrow \phi(s) = \lfloor \alpha, x_1, \dots, x_n \rfloor \wedge n = 1 \wedge M_{\alpha}(x_i) \\
&\quad \text{terminates } \forall i \in \{1, \dots, n\} \\
&\Leftrightarrow \phi(s) \in L_{\text{haltstar}}
\end{aligned}$$

Final mark.

This sequence of double implications is extremely pedantic, but we need it to convince us that we are not cheating. Therefore, we started from the more general undecidable problem L_{halt} coming to the less general, proving that also the more specific one L_{haltstar} is undecidable.

$inc(x)$ function

Problem.

Show that the function $inc : N \rightarrow N$ such that $inc(x) = x + 1$ can be computed by a Turing Machine and it works in linear time.

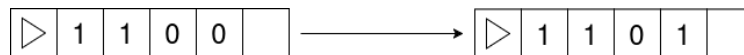
Note.

- Encoding in a Turing Machine works on strings and the given function works within natural numbers.
- We have to build a Turing Machine, since it's intrinsically asked.

Observations.

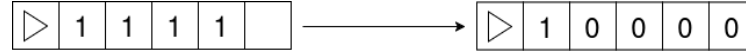
Let's take a look about how the machine behaves before the solution.

Like the image below, we start from the right-side tape and we would like something as on the left-side tape.



Building such a Turing Machine it's quite easy: we move the tape head to right, until it reaches the last binary symbol and we can start encrease by 1.

However, if we suppose a little bit more annoying binary number as follows:

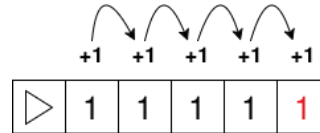


the construction of the Turing Machine seems to be more difficult than before.

What shall we do?

1. Insist with this kind of encoding.
2. Use the **reverse encoding**.

For instance, by reverse encoding the first binary number 1100 would be 0011 and just modifying the first occurence 0, turning it to 1, we could terminate the computation. In the second case the reverse encoding doesn't help us, since 1111 would remain unchanged. Anyway, reasoning with this approach, from the left to the right, we know for sure that we have something to carry until we reach a blank cell.



Solution.

Any possible Turing Machine is described by the triple $M = (\Gamma, Q, \delta)$, where each of them in this solution are composed by:

- $Q = \{q_{init}, q_0, q_1, q_{halt}\}$
- $\Gamma = \{\square, \triangleright, 0, 1\}$
- $\delta =$
 - $(q_{init}, \triangleright) \rightarrow (q_0, \triangleright, R)$
 - $(q_0, 0) \rightarrow (q_1, 1, L)$
 - $(q_0, 1) \rightarrow (q_0, 0, R)$
 - $(q_0, \square) \rightarrow (q_1, 1, L)$
 - $(q_1, 0) \rightarrow (q_1, 0, L)$
 - $(q_1, 1) \rightarrow (q_1, 1, L)$
 - $(q_1, \triangleright) \rightarrow (q_{halt}, \triangleright, S)$
 - $(q_{halt}, \triangleright) \rightarrow (q_{halt}, \triangleright, S)$

where:

- $q_0 \rightarrow$ state before changing the binary number
- $q_1 \rightarrow$ state after changing the binary number

Observations.

- Some transitions are not listed, for instance $(q_{halt}, 0)$ or $(q_1, \square)$, we didn't specify them. These more precise cases seem to be more harmless: even though we describe in details every single transition, our machine still works as before. Therefore, we can suppose that these additional cases will never occur.
- Some states are meaningful only when they are related to specific symbols (*e.g.* q_{init} *within the starting symbol* $\triangleright$).

Chapter 3

Polynomial time computable problems

Computational task as a function

Problem.

Prove that function *minmax* which given (the encoding of) a list of natural numbers $(a_1, \dots, a_m)$ returns both *min* and *max* between $a_1, \dots, a_m$ in **FP**.

Note.

We solve a computational task as a function not as a decision problem (*language*), since we need a *path* to achieve the solution.

Solution.

We can describe an algorithmic solution to this problem as follows:

input: $(a_1, \dots, a_m)$, $m \in N$, $a_i \in N$.

output: (b, c) , where b is the *min* between $a_1, \dots, a_m$ while c is the max value of the list.

Pseudocode.

```
min  $\leftarrow a_1$ 
max  $\leftarrow a_2$ 
for  $i \leftarrow 2$  to  $m$  do
  if  $a_i \leq \textit{min}$  then
     $\textit{min} \leftarrow a_i$ 
  else if  $a_i \geq \textit{max}$  then
```

```

         $max \leftarrow a_i$ 
    end if
end for
return( $min, max$ )

```

Now we state the *three things* to prove that the algorithmic solution is in **FP**. Notice, the *three things* correspond to:

1. Determine the total time complexity of every step done in the solution.
2. Define if intermediate results have a complexity greater or lower than the total one.
3. Auxiliary functions or operations don't have a time complexity greater than the total one derived.

1st Total number of steps.

We are gonna to take the whole pseudocode and analyze it piece by piece, considering one macro-operation at time.

1st macro-operation $\rightarrow$ assignment done for 2 times.

```

 $min \leftarrow a_1$ 
 $max \leftarrow a_2$ 

```

2nd macro-operation $\rightarrow$ loop done for n times (it can be described as $n - 1$ since the loop starts from 2 and not 1, but we can already say that it's proportional to $O(n)$).

```

for  $i \leftarrow 2$  to  $m$  do
end for

```

3rd macro-operation $\rightarrow$ repeating comparisons and assignments at most 4 times at each iteration.

```

if  $a_i \leq min$  then
     $min \leftarrow a_i$ 
else if  $a_i \geq max$  then
     $max \leftarrow a_i$ 
end if

```

4th macro-operation $\rightarrow$ return done 1 time.

```

return( $min, max$ )

```

Therefore, the pseudocode above executes at most:

$$(2 + O(n) \cdot 4 + 1) \in O(n)$$

We can also state that the complexity of the algorithm is $\Theta(n)$, since we have to scan the whole list in any circumstance, so it's strictly proportional to n .

2nd Intermediate results.

The only intermediate results are *min* and *max*, not to much designing this algorithm. However, we can notice that the sizes of *min* and *max* are never greater than the length of the input list, in particular:

- $|min| \leq |a_1, \dots, a_m| \leq n$
- $|max| \leq |a_1, \dots, a_m| \leq n$
- $|i| \leq \log(m) \in O(m)$

We are just stressing that all the intermediate results belong to $O(n)$ or, better, they have in the worst case a polynomial time complexity.

3rd Auxiliary functions.

The auxiliary functions are:

- two comparisons;
- assignments done over the index.

Each instruction can itself be executed in polynomial time, since the only non-elementary instruction done is the comparison.

Final mark.

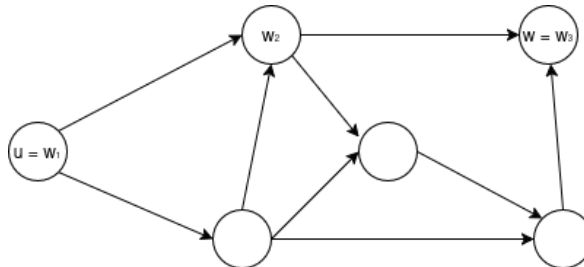
All in all, given the previous three points, we can state that *minmax* function is bounded by polynomial time execution, and:

$$minmax \in \mathbf{FP}$$

Decision problem about graphs

Problem.

If $G = (V, E)$ is a graph, the reachability problem asks to determine, given $u, v \in V$, whether v is reachable from u , namely whether exists a sequence $w_1, \dots, w_n$ of nodes with $u = w_1$, $v = w_n$ and $(w_i, w_{i+1}) \in E$ for every $i \in \{1, \dots, n-1\}$.



Prove that the above **decision problem** is in **P**.

Solution.

First of all, we define the waited input and the expected output from the algorithmic solution, in particular:

input: $G = (V, E)$, u , v .

output: 0 or 1 depending on v is reachable from u .

Pseudocode.

```
visited  $\leftarrow [u]$ 
current  $\leftarrow [u]$ 
while current  $\neq []$  do
    pick  $w$  from current
    remove  $w$  from current
    for each  $(w, t) \in E$  do
        if  $t \notin \text{visited}$  then
            add  $t$  to visited
            add  $t$  to current
        end if
    end for
end while
if  $v \in \text{visited}$  then
    return true
else
    return false
end if
```

Note.

It's not a computational task described as a function, since we provide if v belongs or not to the list *visited*.

Before stating the *three things* in order to prove the time complexity of this solution, we could describe some observations about it.

Observations.

- *Visited* list can only grow and never shrink.
- We use the double-list check to avoid infinite loops if the graph contains cycles.
- *Current* list is always a sublist of *visited* list, whatever we have in

current it's also inside *visited*.

- Whenever we add a node in *current* we do this if and only if $t \notin \textit{visited}$.
As a consequence, it could never be part of *current* in the past.

Now we state the *three things* to prove that the algorithmic solution above is in **P**.

1st Total number of steps.

We take the whole pseudocode and we analyze it piece by piece, considering one macro-operation at time.

1st macro-operation $\rightarrow$ while loop done for n times.

```
while current  $\neq$  [ ] do
end while
```

2nd macro-operation $\rightarrow$ pick and remove a node done at most 2 times.

```
pick  $w$  from current
remove  $w$  from current
```

3rd macro-operation $\rightarrow$ for loop done for n times.

```
for each  $(w, t) \in E$  do
end for
```

4th macro-operation $\rightarrow$ comparison and lists manipulation done at most 3 times.

```
if  $t \notin \textit{visited}$  then
    add  $t$  to visited
    add  $t$  to current
end if
```

5th macro-operation $\rightarrow$ comparison and return done at most 4 times.

```
if  $v \in \textit{visited}$  then
    return true
else
    return false
end if
```

Therefore, the pseudocode above executes at most:

$$O(n) \cdot (2 + O(n) \cdot 3) + 4 \in O(n^2)$$

2nd Intermediate results.

There are just two intermediate variables, *visited* and *current*, both containing vertices without any possibility of inner duplications. Therefore, their sizes are clearly polynomial in n .

3rd Auxiliary functions.

Each instruction can be executed in polynomial time. All of them are list manipulation operations:

1. add element;
2. remove element;
3. check if there's an element in the list.

For sure, every instruction or manipulation operation about a list can be executed in polynomial time.

Final mark.

Summing up, the reachability problem is in **P**.